

Scalability Design Note

Project: AI Legal Contract Intelligence Assistant

Objective: Architecture evolution strategy to scale from a local prototype to an enterprise-grade cloud deployment.

Scaling a Retrieval-Augmented Generation (RAG) system from handling 100 papers to an enterprise-grade system handling 100,000+ papers require a shift from synchronous, single-node processing to an asynchronous, distributed, and highly available architecture.

Below is the strategic roadmap for evolving each layer of the system.

1. Chunking Strategy

- **Current (100 Papers):** Basic recursive character splitting with fixed overlaps.
- **Scale (100,000 Papers):** Semantic and structural chunking. We will implement document-aware parsers that respect headers, paragraphs, and sections. This prevents the system from blindly splitting sentences in half and ensures that highly relevant contextual boundaries are preserved, which drastically improves retrieval accuracy over a massive corpus.

2. Embedding Generation

- **Current (100 Papers):** Synchronous embedding generation on the API server's CPU.
- **Scale (100,000 Papers):** Dedicated GPU-accelerated embedding endpoints. We will offload the creation of embeddings to a dedicated service running on GPU instances (e.g., AWS SageMaker or custom instances) to handle massive batch jobs. This prevents the main web server from locking up during the ingestion of large PDF batches.

3. Background Processing

- **Current (100 Papers):** File uploads and text processing block the HTTP request until completion.
- **Scale (100,000 Papers):** Event-driven asynchronous architecture. Document uploads will be sent directly to an object store (like Amazon S3). S3 events will trigger messages in a message broker (such as Apache Kafka or RabbitMQ). A fleet of background worker nodes (e.g., Celery) will consume these messages, process the PDFs, generate embeddings, and update the database asynchronously without making the user wait.

4. Vector Database Choice

- **Current (100 Papers):** Local, file-based vector storage (e.g., ChromaDB or FAISS local files).
- **Scale (100,000 Papers):** Managed, distributed vector database. We will migrate to an enterprise solution like Pinecone, Milvus, or a managed Qdrant cluster. At 100,000 papers, the system will manage tens of millions of embeddings. These distributed databases handle horizontal sharding, guarantee high availability, and allow for crucial **metadata filtering** (filtering by author, date, or topic before running the vector search) to keep retrieval times under 100ms.

5. Caching

- **Current (100 Papers):** No caching; every query hits the database and the LLM.
- **Scale (100,000 Papers):** Multi-tier caching.
 - **Semantic Caching:** We will use a tool like GPTCache backed by Redis. If a user asks a question that is semantically identical to a previous query (e.g., “What is the conclusion of paper X?” vs. “Summarize paper X’s conclusion”), the system returns the cached AI response instantly, bypassing both the vector search and the LLM API.
 - **Standard Caching:** Redis will also cache raw document metadata and frequent database lookups.

6. API Performance

- **Current (100 Papers):** Single web server instance.
- **Scale (100,000 Papers):** Load-balanced, stateless microservices. The backend (FastAPI/Node) will be completely stateless. An Application Load Balancer (ALB) will distribute traffic across multiple API instances. We will utilize connection pooling for databases and gRPC for fast internal communication between microservices.

7. Deployment Scaling

- **Current (100 Papers):** Local machine or a single monolithic Virtual Machine.
- **Scale (100,000 Papers):** Containerized orchestration via Kubernetes (K8s). All services (API, workers, embedding generators) will be Dockerized. Kubernetes will manage the deployment, automatically restarting failed containers and using Horizontal Pod Autoscalers (HPA) to spin up additional background workers automatically when there is a massive spike in document uploads.

8. Monitoring and Logging

- **Current (100 Papers):** Terminal console logs.
- **Scale (100,000 Papers):** Centralized observability stack.
 - **Logging:** All container logs will be streamed to an ELK stack (Elasticsearch, Logstash, Kibana) or Datadog for centralized debugging.
 - **Metrics:** Prometheus and Grafana will track system health (CPU/Memory usage, queue lengths, HTTP 500 errors).
 - **Tracing:** OpenTelemetry will be used to track the exact latency of every step in the pipeline—from the moment a user asks a question, through the vector search, to the final LLM response time.

9. Cost Considerations

- **Storage Tiering:** Raw PDFs will be moved to cold storage (like S3 Glacier) after a period of inactivity to save costs, keeping only the embeddings in the high-performance database.
- **Compute Optimization:** Background worker nodes will utilize Spot Instances (spare cloud compute capacity), which can reduce ingestion compute costs by up to 70%.
- **LLM Token Management:** We will implement strict token limits and dynamic prompt sizing to ensure we do not overpay for cloud LLM API calls. For internal summary tasks, we may route requests to smaller, cheaper open-source models (like LLaMA 3 8B), reserving premium, expensive models only for complex, nuanced reasoning.